

RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO PRÁTICA DE CONEXÕES SEGURAS

Este relatório técnico documenta as Provas de Conceito (PoCs) desenvolvidas para a homologação dos mecanismos de segurança de redes e camada de aplicação, atestando a vulnerabilidade de protocolos legados e a eficácia das arquiteturas modernas de proteção (TLS e Gestão Segura de Sessões). O ambiente de laboratório utilizado consistiu em uma instância Ubuntu Server 22.04 LTS provisionada na infraestrutura da AWS (Cloud9/EC2).

MÓDULO 1: SNIFFING E INTERCEPTAÇÃO (CAMADA 2 E 3)

Ferramenta Usada: iproute2 (comando ip), tcpdump e curl.

Passo a Passo:

1. Instalação das dependências no ambiente via `sudo apt update && sudo apt install tcpdump curl -y`.
2. Identificação da interface de rede principal (ens5).
3. Habilitação do modo promíscuo no Controlador de Interface de Rede (NIC) através do comando: `sudo ip link set dev ens5 promisc on`.
4. Inicialização do *sniffer* de rede para captura de tráfego HTTP na porta 80, exibindo o *payload* em ASCII: `sudo tcpdump -i ens5 -nn -A -s 0 port 80`.
5. Em um terminal secundário, injeção de tráfego não criptografado realizando uma requisição via comando: `curl http://neverssl.com`.
6. Após a captura das evidências, a interface retornou ao estado original com `sudo ip link set dev ens5 promisc off`.

Resultado:

O *sniffer* (tcpdump) exibiu imediatamente o fluxo de dados em texto legível. A captura comprovou a interceptação do cabeçalho da requisição (GET / HTTP/1.1) e do corpo da resposta, contendo as tags do código-fonte HTML do servidor de destino. (*Vide print de evidência no anexo do repositório*).

Explicação Técnica do Resultado:

A operação demonstra que o Modo Promíscuo altera o comportamento padrão da NIC, forçando-a a encaminhar todos os quadros interceptados no segmento de rede para a CPU, ignorando o filtro nativo de endereços MAC. Como o protocolo HTTP não atua sobre uma camada de transporte segura (TLS), os dados são transmitidos em Texto Claro (Plain Text). Isso comprova a extrema vulnerabilidade de comunicações locais não encriptadas frente a ataques

de interceptação na Camada 2 (Data Link).

MÓDULO 2: CRIPTOGRAFIA E HANDSHAKE (A BASE DO TLS)

Ferramenta Usada: OpenSSL.

Passo a Passo:

1. **Geração Assimétrica (Servidor):** Criação de uma chave privada RSA de 2048 bits (`openssl genpkey`) e extração da sua respectiva chave pública (`openssl rsa`).
2. **Geração Simétrica (Cliente):** Criação de uma chave de sessão AES-256 de 32 bytes (`openssl rand`).
3. **Simulação do Handshake:** O cliente utilizou a Chave Pública RSA do servidor para encriptar a Chave Simétrica de sessão, gerando o arquivo protegido `encrypted_symmetric.bin`.
4. **Transferência Segura:** Um arquivo de texto contendo dados sensíveis foi criptografado com a Chave Simétrica (AES-256-CBC), gerando o `secret.enc`.
5. **Recepção:** O servidor utilizou sua Chave Privada RSA para descriptar a chave de sessão do cliente e, de posse dessa chave simétrica, descriptografou os dados e leu o payload intacto.

Resultado:

O servidor conseguiu reconstruir e exibir no console a mensagem exata: *"Dados corporativos extremamente confidenciais - Modulo 2"*. Durante o processo de transferência, todos os arquivos gerados encontravam-se ofuscados em formato binário ilegível. (Vide print de evidência no anexo do repositório).

Explicação Técnica do Resultado:

A Prova de Conceito valida a eficácia da Criptografia Híbrida, arquitetura base do HTTPS. A criptografia assimétrica (RSA) é computacionalmente muito pesada e introduziria alta latência se aplicada a todo o tráfego. Portanto, ela é utilizada exclusivamente na fase de Handshake, com o único propósito de transferir a Chave de Sessão de forma segura. Após essa troca, a comunicação passa a utilizar a criptografia simétrica (AES), que suporta aceleração em hardware e garante uma transferência de dados em altíssima velocidade e segurança.

MÓDULO 3: SEGURANÇA NA CAMADA DE APLICAÇÃO (DESENVOLVIMENTO)

Ferramenta Usada: Node.js, framework web Express.js e curl.

Passo a Passo:

1. Inicialização de um projeto Node.js (npm init -y) e instalação da biblioteca express.
2. Criação de um servidor web local (server.js) escutando na porta 3000.
3. Criação de dois endpoints de roteamento distintos:
 - /login-inseguro: Rota legada que injeta no navegador do usuário um Cookie SessionID sem configurações restritivas.
 - /login-seguro: Rota moderna que injeta o Cookie SessionID utilizando explicitamente as diretivas operacionais { secure: true, httpOnly: true }.
4. Realização de testes de *request* HTTP utilizando o comando curl -v para forçar a exibição detalhada dos cabeçalhos (*headers*) retornados pelo servidor.

Resultado:

Ao consultar a rota insegura, o servidor retornou o cabeçalho explícito: < Set-Cookie: SessionID=abc123vulneravel; Path=/.

Ao consultar a rota segura, o servidor retornou o cabeçalho mitigado: < Set-Cookie: SessionID=xyz987protegido; Path=/; HttpOnly; Secure. (*Vide print de evidência no anexo do repositório*).

Explicação Técnica do Resultado:

O HTTP é um protocolo sem estado (*stateless*). Para gerir autenticações, utiliza-se o Cookie SessionID como um "crachá" de acesso. O risco crítico para a pessoa desenvolvedora é o *Session Hijacking* (Roubo de Sessão). Se o cookie for transmitido via HTTP puro (Rota 1), atacantes que praticam *sniffing* na rede (demonstrado no Módulo 1) podem interceptá-lo e assumir o controle da conta.

O papel fundamental do HTTPS na mitigação desse risco (Rota 2) ocorre devido à flag Secure. Esta flag instrui os navegadores a bloquearem o envio do Cookie a menos que o tráfego esteja encapsulado sob a criptografia do TLS, neutralizando a interceptação física. Aliado a isso, a flag HttpOnly neutraliza roubos de sessão baseados em injeção de scripts (XSS), fornecendo uma defesa em profundidade.

Fim do Documento